

从Chatbot到Multi-Agent

用Agent Loop与MCP协议构建和组合的AI开发 workflows

郑 予 彬

资深开发者布道师 亚马逊云科技

2025/00/00



CONTENT

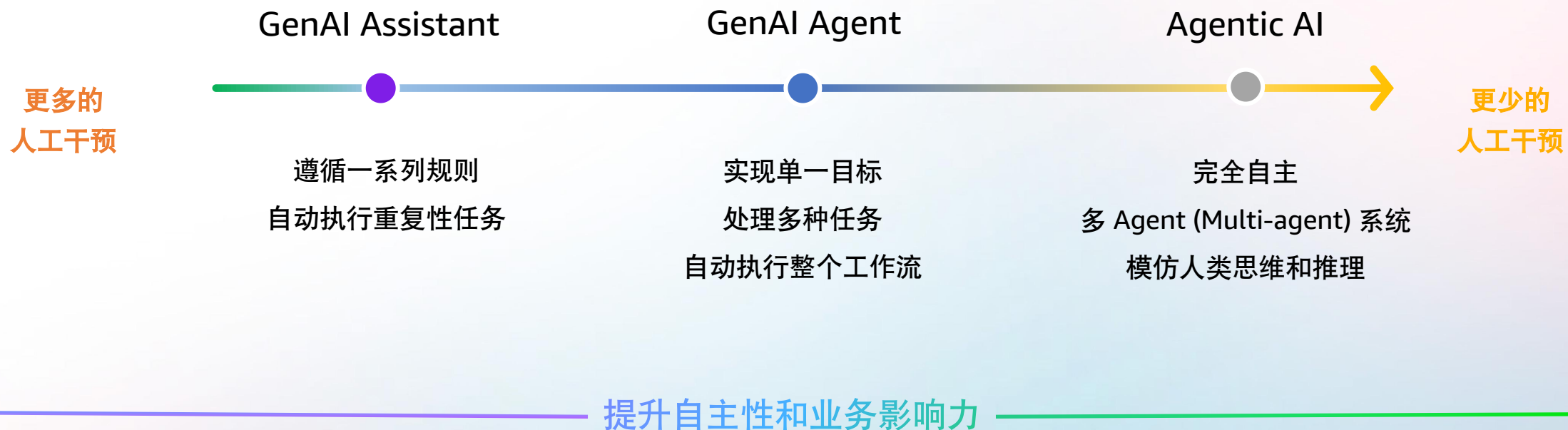
目录

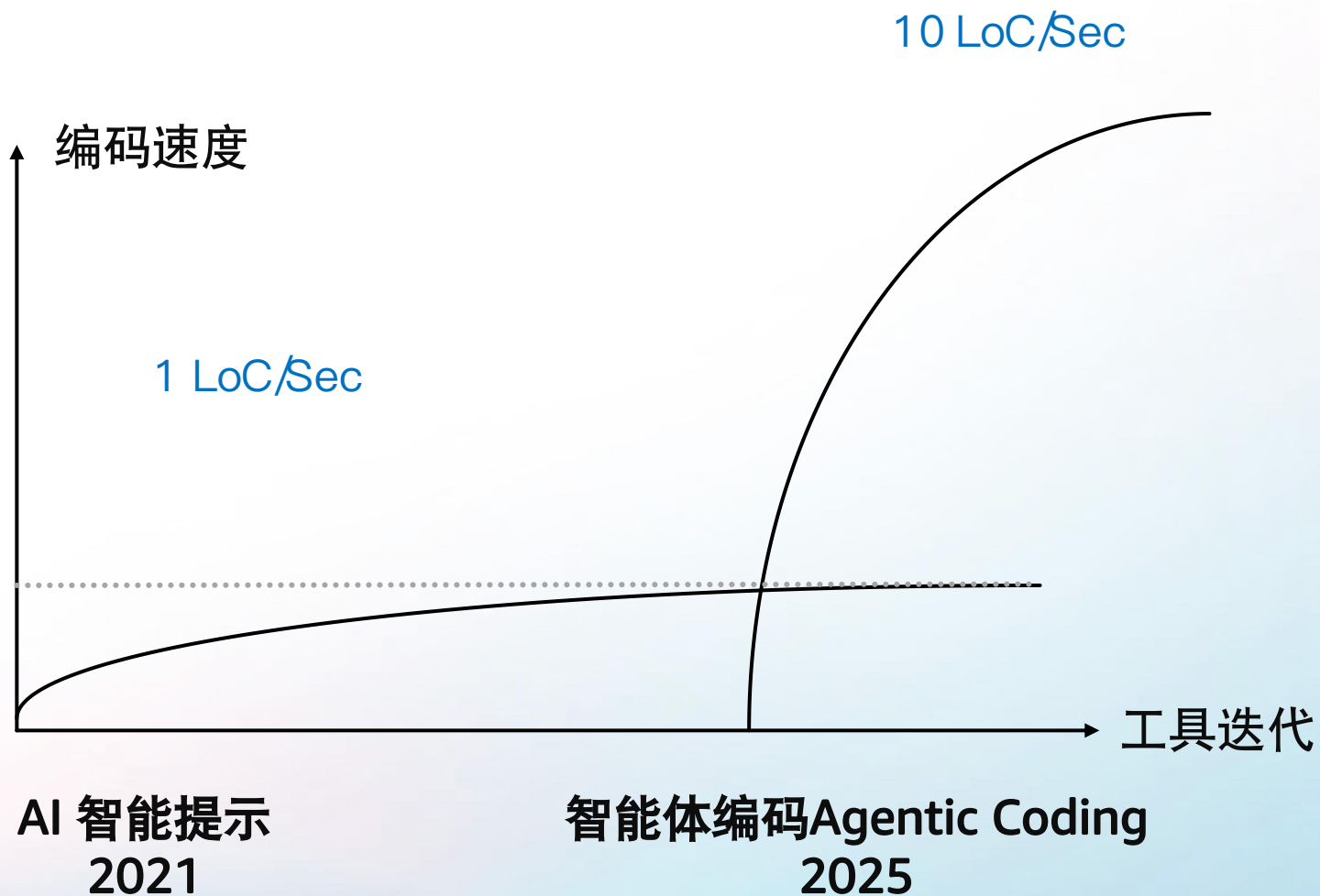
- 01 从单点智能到系统化AI Agent
- 02 适用于生产的规格化开发
- 03 现代开发范式

从单点智能到系统化AI Agent



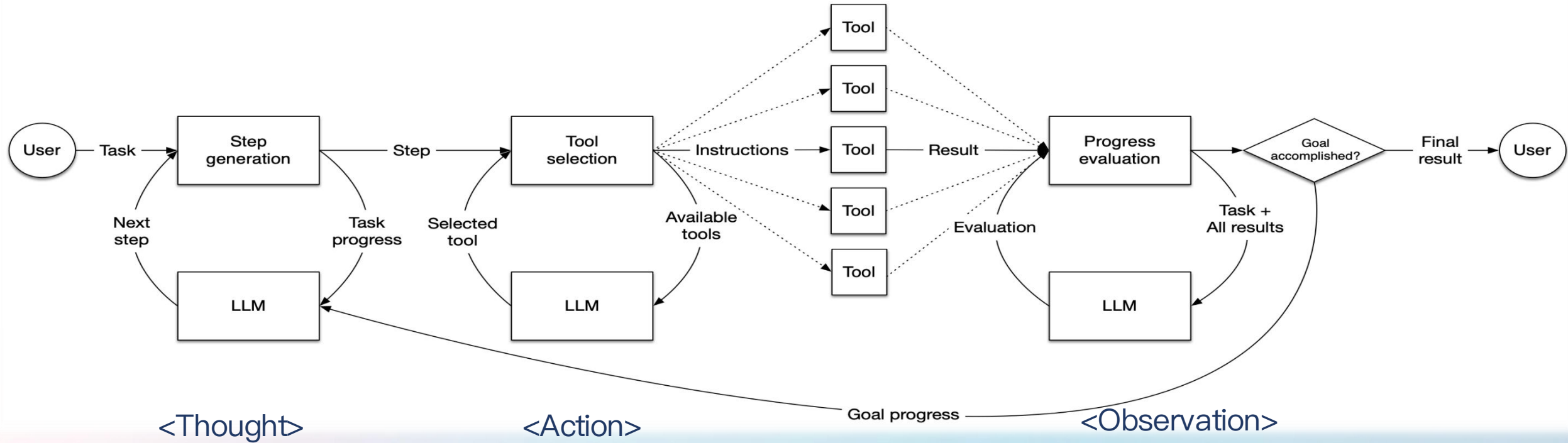
Agentic AI 的发展历程





- 2021: AI 智能提示, 代码自动补全
- 2024: 生成更大的代码段并回答问题
- 2025: 智能体编程 (Agentic coding) 可以带来数十倍的编码速度提升

ReAct Agent

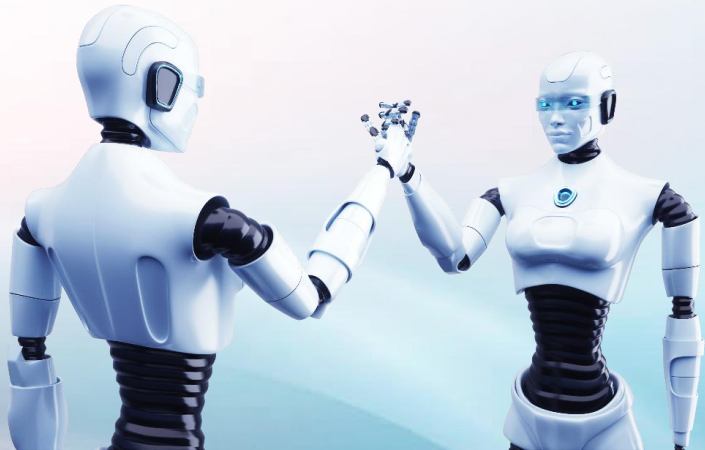


《ReAct: Synergizing Reasoning and Acting in Language Models》

Modern Agent

在实际工程中，ReAct 被扩展了，Modern Agent 在 ReAct 基础上增加了：

- 任务分解(Task decomposition)
- 工具管理 (Tool selection, routing)
- 长期记忆 (Memory)
- 多 Agent 协作 (Multi-agent)
- 进度评估与控制 (Evaluation & Feedback loops)



Strands Agents是一个开源SDK，只需几行代码即可构建AI Agent



通过**Agent 循环 (Agentic Loop)**，充分利用了现代**LLM**的推理、规划、调用工具和自我反思的能力，简化了**Agent**开发过程。



内置工具集、**MCP Server**，以及与**Amazon Web Services**服务的原生集成



支持自定义模型



支持多种**multi-Agent**构建模式

Agent Loop

让Agent 能够执行复杂、多步骤的推理和行动



Agent Loop

Agents are models using tools in a loop

```
env = Environment()
tools = Tools(env)
system_prompt = "Goals, constraints, and how to act"

while True:
    action = llm.run(system_prompt + env.state)
    env.state = tools.run(action)
```

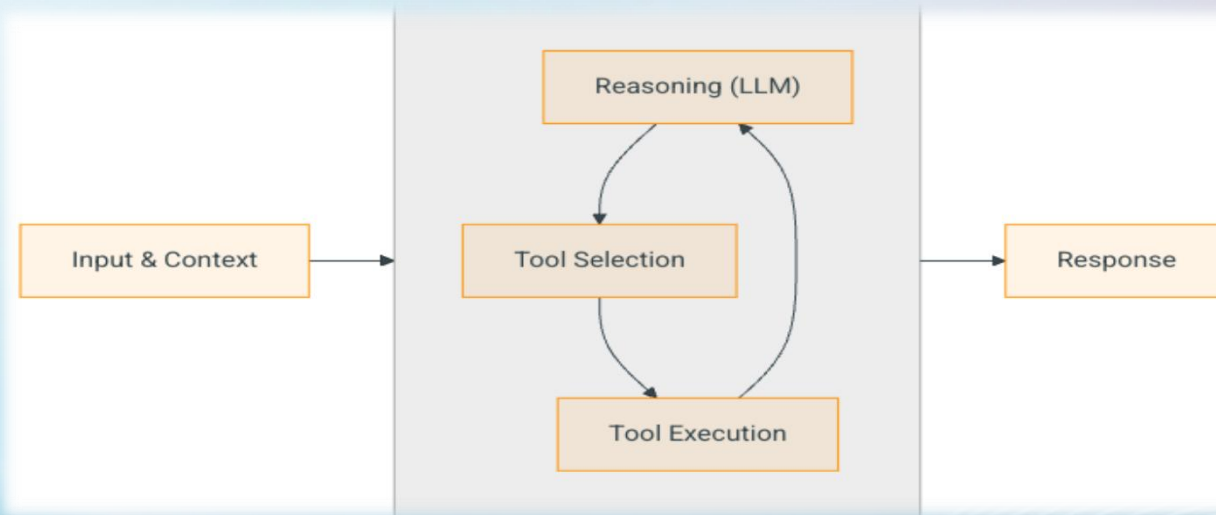
Agent自主进行规划和操作，必要时会向用户寻求更多信息或判断。

在执行过程中，**Agent**需要在每一步都从环境中获取“反馈”（如工具调用结果或代码执行情况）来完成自我迭代，直至一次输入任务完成，或者达到最大迭代次数

Agent视为"在循环中使用工具的模型"

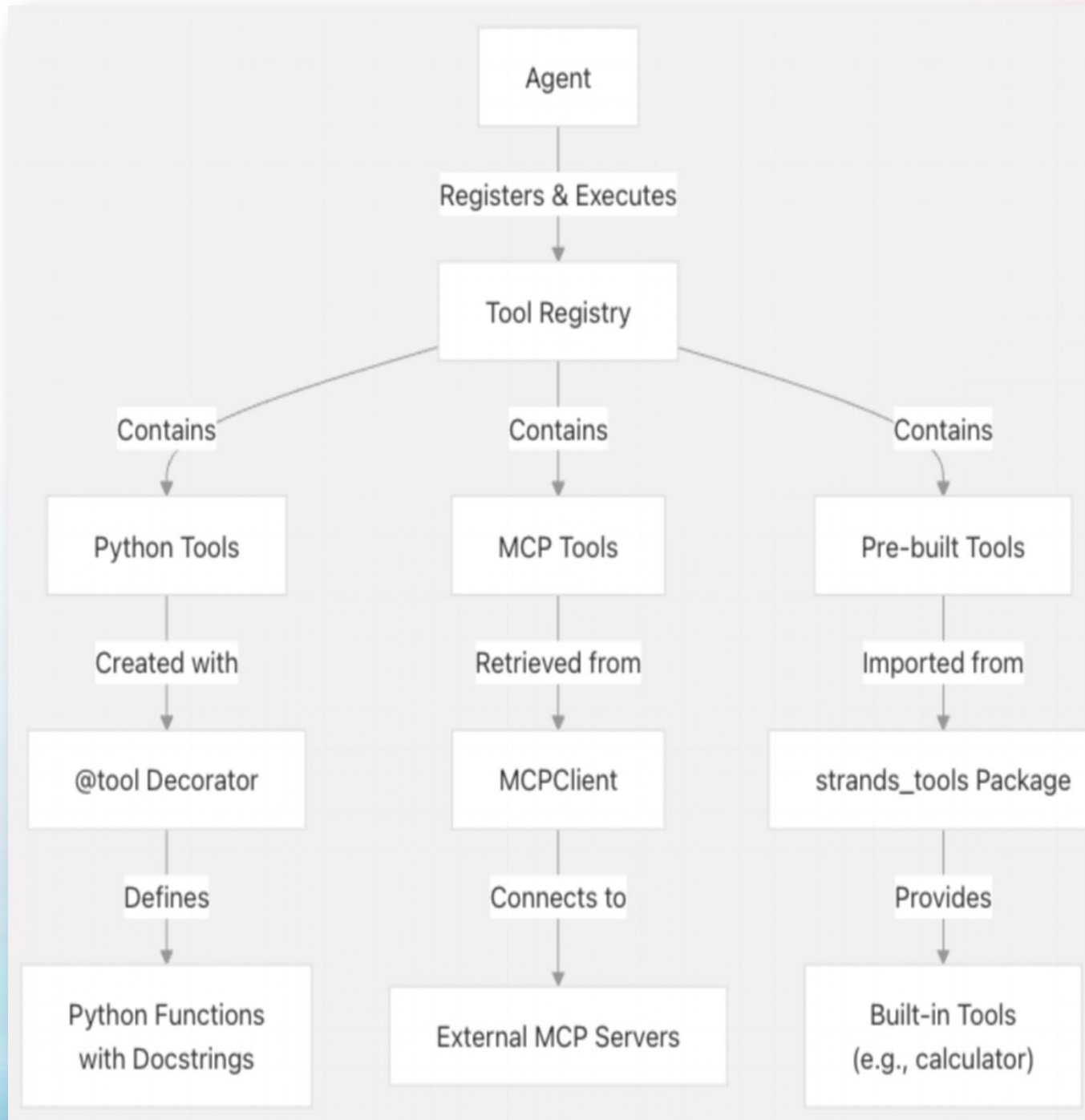
核心组成部分：

- 环境（**Agent**所处的上下文空间）
- 工具（用于交互和获取反馈）
- 系统提示（定义目标与约束）



内置工具和接口

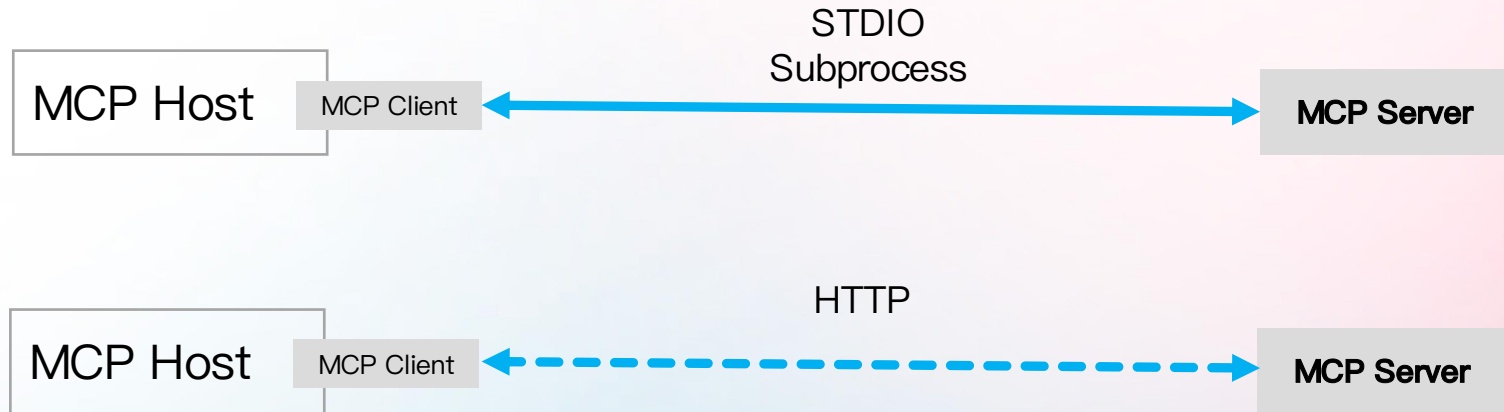
- **Python Tools(自定义):** 通过简单的`@tool`装饰器标记Python函数，使agents能够直接调用这些函数，实现定制化功能扩展。
- **MCP Tools:** 借助CMP协议访问的外部工具集，让您的agents能够与更广泛的生态系统无缝集成
- **Pre-built Tools:** 从`strands\_tools`包中直接使用的即用型工具，无需额外开发，快速增强agents能力。



MCP Server的访问方式

本地MCP server

- 作为MCP Client 子进程加载，采用标准输入输出传输
- Client-Server 1:1绑定
- 通过环境变量传递credentials



Remote MCP server

- 部署在远程Http Server上
- 采用Streamable HTTP传输
- 通过Oauth2.1鉴权
- 支持多租户

For HTTP-based MCP servers that use Streamable-HTTP Events transport:

```
from mcp.client.streamable_http import streamablehttp_client
from strands import Agent
from strands.tools.mcp.mcp_client import MCPClient

streamable_http_mcp_client = MCPClient(lambda: streamablehttp_client("http://localhost:8000/mcp"))

# Create an agent with MCP tools
with streamable_http_mcp_client:
    # Get the tools from the MCP server
    tools = streamable_http_mcp_client.list_tools_sync()

    # Create an agent with these tools
    agent = Agent(tools=tools)
```

内置工具集

- 文件操作 –读取、写入和编辑文件，具有语法高亮和智能修改功能
- Shell集成 –安全地执行和交互shell命令
- 记忆工具 –在Agent运行过程中存储用户和代理记忆，包括Mem0和KB两种方案
- HTTP客户端 –进行API请求，支持身份验证
- Python执行 –运行Python代码片段，具有状态持久性、用户代码执行确认和安全功能
- 数学工具 –使用符号数学功能执行高级计算
- AWS集成 –通用性的AWS服务访问工具，如S3, EC2, DDB等
- 图像处理 –为AI应用生成和处理图像
- 环境管理 –安全处理环境变量
- 日志记录 –创建和管理结构化日志和日记
- 高级推理 –复杂思考和推理能力的工具
- Swarm群体智能 –协调多个Agents进行并行问题解决，共享内存
- Graph/Workflow 工具 –创建Agent graph/workflow的工具

多模型支持

- Amazon Bedrock 模型
- Anthropic API
- Llama API
- Ollama
- OPENAI 及兼容
- 通过集成 LiteLLM 支持的其他模型，例如：硅基流动

LiteLLM

```
from strands import Agent
from strands.models.litellm import LiteLLMModel
from strands_tools import calculator

model = LiteLLMModel(
    # **model_config
    client_args={
        "api_key": "sk-a[REDACTED]xhk",
        "base_url": "https://api.siliconflow.cn/"
    },
    model_id="openai/Qwen/Qwen3-235B-A22B",
    params={
        "max_tokens": 1000,
        "temperature": 0.7,
    }
)

agent = Agent(model=model, tools=[calculator])
response = agent("What is 2+2")
print(response)
```

OPENAI 兼容

```
from strands.models.openai import OpenAIModel
from strands_tools import calculator
import os

model = OpenAIModel(
    # **model_config
    client_args=[
        "api_key": os.environ["OPENAI_API_KEY"],
        "base_url": "https://api.siliconflow.cn/"
    ],
    model_id="Pro/deepseek-ai/DeepSeek-V3",
    max_tokens = 1000,
    temperature = 0.7,
)

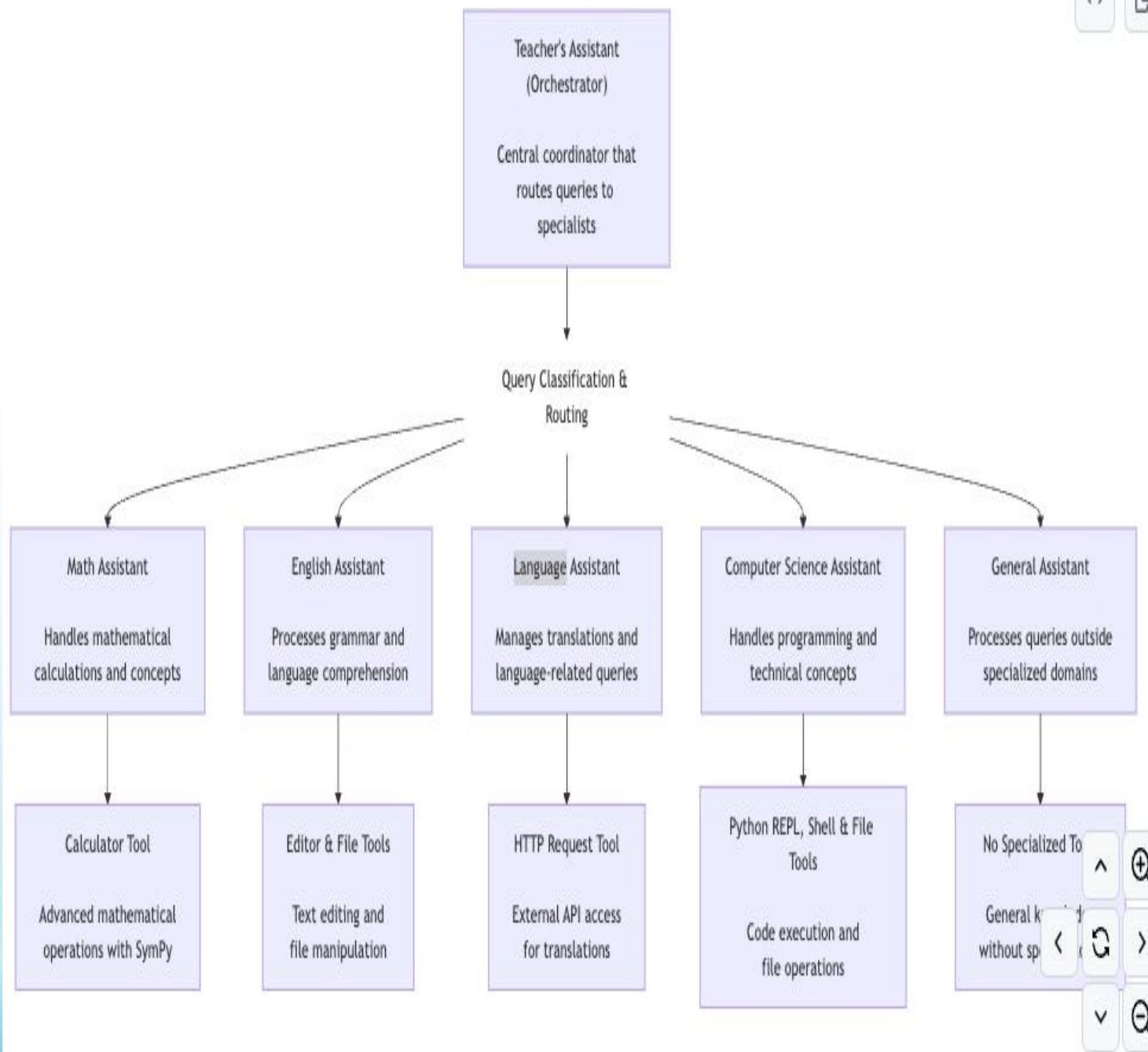
agent = Agent(model=model, tools=[calculator])
response = agent("what is 2+2")
```

Multi Agen 构建模式

Multi-agent模式	描述	相似类比
Agent as Toos （Orchestrator）	Lead-agent 协调多个Sub-agent， Sub-agent 作为Lead-agent的工具使用	• Amazon Bedrock Multi-agent • Multi-agent ochestrator • OpenAI agent sdk handoff
Swarm/群体智能	通过共享信息和context，把任务分给多个Agent并行解决问题	Autogen groupchat
Graph	具有网络拓扑结构的Agent互联网络， Node 代表Agent， Edge代表Agent之间的通信	LangGraph
Workflow	通过固定工作流控制Agent信息和执行顺序， 一个Agent的输出是下一个Agent的输入 context	LangGraph

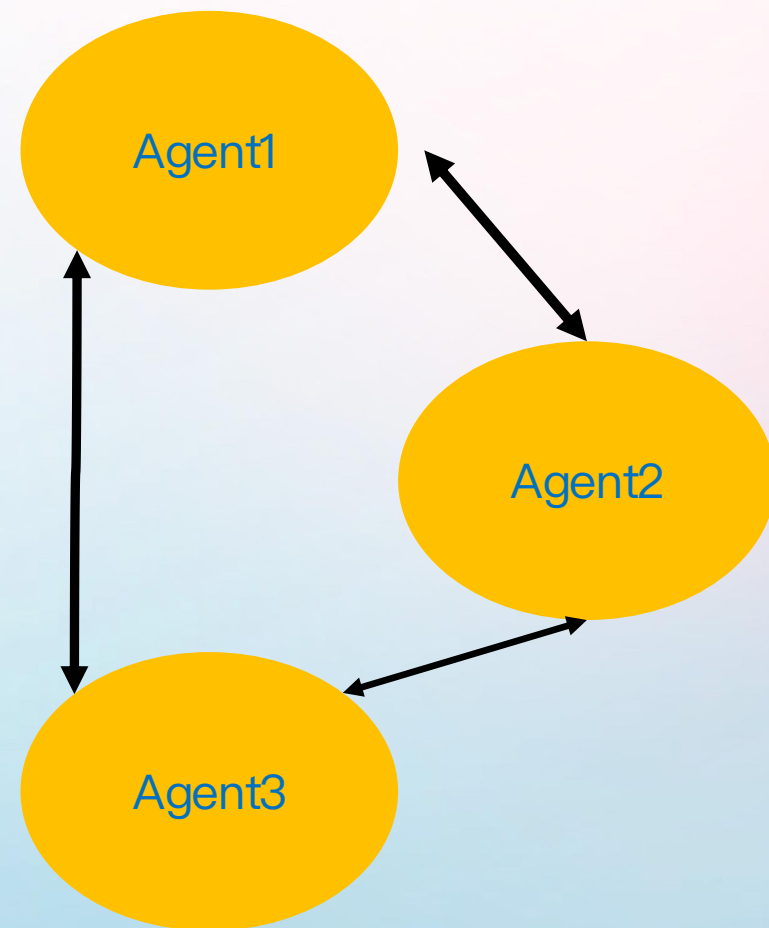
Orchestrator 模式 (Agents as tools)

- 分而治之：每个专家Agent都有明确的责任领域，使系统更容易理解和维护
- 中央集权，层级委派：协调者决定调用哪个专家Agent，形成明确的指挥链条
- 模块化架构：Agent专家可以独立优化，添加、删除或修改，甚至复用到其他的系统中



Swarm 群体智能

- 去中心化：每个Agent相对独立且并行执行任务，没有上级指挥。单个Agent 相对简单，通过群体协作达成更高难度任务
- 上下文信息共享：每个Agent都需要看到其他Agent的输入输出信息
- 多种协作方式： Collaborative, Competitive, Hybrid



示例：使用swarm tool动态创建multi agents

Collaborative 模式下，每个Agent会在其他Agent的基础上进行完善

```
agent = Agent(tools=[swarm])
result = agent.tool.swarm(
    task="AI对人类发展的影响是什么？",
    system_prompt="你是一个善于思辨的哲学家",
    swarm_size=4,
    coordination_pattern="collaborative",
)
```

```
{'text': '🤖 Agent agent_1: Response: # AI对人类发展的影响: 综合视角\n\n分析前几位专家的观点, 我看到了我们对AI影响的多层次认识。我想在此基\n{'text': '🤖 Agent agent_1: Metrics: Event Loop Metrics Summary:\n| Cycles: total=1, avg_time=17.135s, total_time=17.135s\n{'text': '🤖 Agent agent_2: Response: # AI对人类发展的影响: 整合与扩展视角\n\n我想基于前一阶段的讨论, 对AI对人类发展影响的分析进行整合与扩\n{'text': '🤖 Agent agent_2: Metrics: Event Loop Metrics Summary:\n| Cycles: total=1, avg_time=23.725s, total_time=23.725s\n{'text': '🤖 Agent agent_3: Response: # AI对人类发展的影响: 整合视角\n\n基于之前的讨论, 我想从以下几个角度进一步整合与深化AI对人类发展影响\n{'text': '🤖 Agent agent_3: Metrics: Event Loop Metrics Summary:\n| Cycles: total=1, avg_time=57.481s, total_time=57.481s\n{'text': '🤖 Agent agent_4: Response: # AI对人类发展的综合影响分析\n\n我将整合并深化前一阶段讨论的洞见, 提供对AI影响人类发展的更全面思考。\n{'text': '🤖 Agent agent_4: Metrics: Event Loop Metrics Summary:\n| Cycles: total=1, avg_time=21.360s, total_time=21.360s\n{'text': '\n🌟 Collective Knowledge:\n\n { "id": "3f0dba92-e10c-4005-8d83-9ee826bf7644", "agent_id": "agent_3"
```

Competitive 模式下，每个Agent会在其他Agent的基础上补充其他的视角

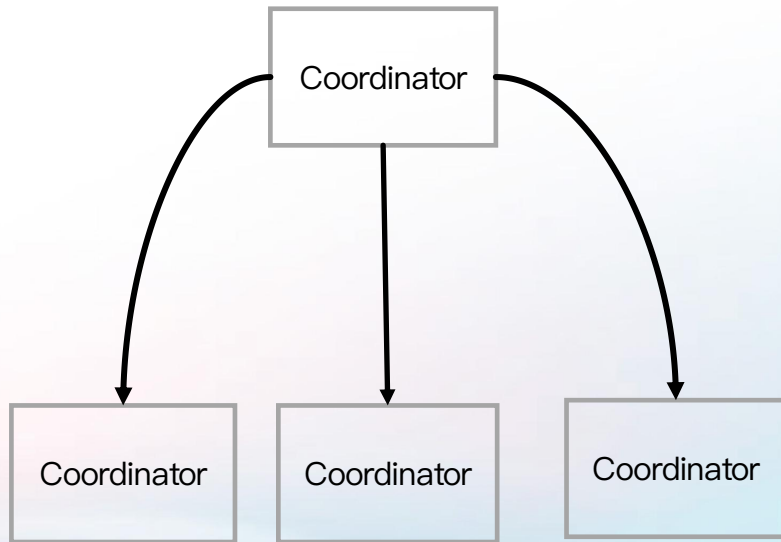
```
result = agent.tool.swarm(
    task="AI对人类发展的影响是什么？",
    system_prompt="你是一个善于思辨的哲学家",
    swarm_size=4,
    coordination_pattern="competitive",
)
```

```
角\n\n基于前一阶段的讨论, 我要从长期共生演化角度分析AI对人类发展的影响, 这是一个之前未被充分探讨的维度:\n\n## 人类认知与AI系统的协同进化\n\n人类认\n- Cycles: total=1, avg_time=19.229s, total_time=19.229s\n| Tokens: in=7641, out=539, total=8180\n| Bedrock Latency: 18908ms\n分配视角\n\n在前一轮讨论中, 我们看到了从生态系统、经济效益、认知重构和社会结构等多个角度对AI影响的分析。我想提供一个独特的补充视角: AI如何重塑全球资源\n- Cycles: total=1, avg_time=50.601s, total_time=50.601s\n| Tokens: in=7641, out=644, total=8285\n| Bedrock Latency: 16762ms\n重塑视角\n\n根据以往的分析, AI对人类发展的影响已从技术、经济、社会结构等多个维度进行了探讨, 但对伦理与价值观的重塑尚未深入。从这一独特视角出发:\n\n# 价值\n- Cycles: total=1, avg_time=24.212s, total_time=24.212s\n| Tokens: in=7641, out=564, total=8205\n| Bedrock Latency: 23884ms\n转型视角\n\n我将从伦理与价值观转型的独特视角探讨AI对人类发展的影响, 这是前面分析中尚未充分展开的领域:\n\n## 价值观体系的重构\n\nAI系统内嵌的价值取\n- Cycles: total=1, avg_time=18.695s, total_time=18.695s\n| Tokens: in=7641, out=483, total=8124\n| Bedrock Latency: 18386ms
```

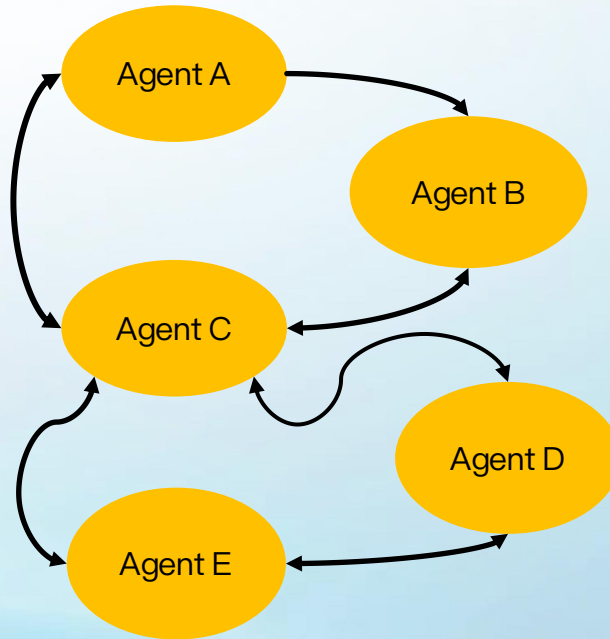

Agent Graph

- 利用Agent as Tools 机制组成更复杂的Agent 网络
- 支持拓扑类型: star, mesh, hierarchical

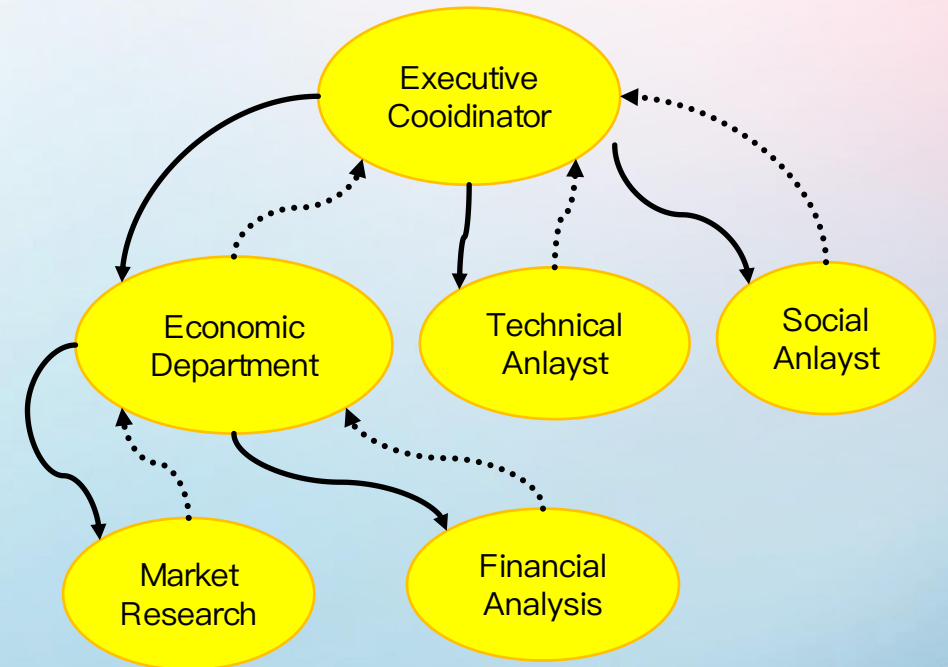
Star/星型



Mesh/网状



Hierarchical/分层



示例：使用Agent graph创建multi agents

方法1:使用自然语言动态创建Agent graph

```
from strands import Agent
from strands_tools import agent_graph
# Initialize Agent with agent_graph
agent = Agent(tools=[agent_graph])

# Create a network through natural language
agent("Create a research team with a coordinator, data analyst, and domain expert in a star topology")

# Send a task to the team
agent("Ask the research team to analyze emerging technology adoption in healthcare")

# Check status and manage the network
agent("Show me the status of the research team")
```

```
agent("Show me the status of the research team")
# Check status and manage the network
```

Code sample

https://github.com/strands-agents/tools/blob/main/src/strands_tools/agent_graph.py

这个工具可以让agent自行创建Sub-Agent并组成网络

```
TOOL_SPEC = {
    "name": "agent_graph",
    "description": """Create and manage graphs of agents with diff
```

Key Features:

1. Multiple topology support (star, mesh, hierarchical)
2. Dynamic message routing
3. Parallel agent execution
4. Real-time status monitoring
5. Flexible agent configuration

Example Usage:

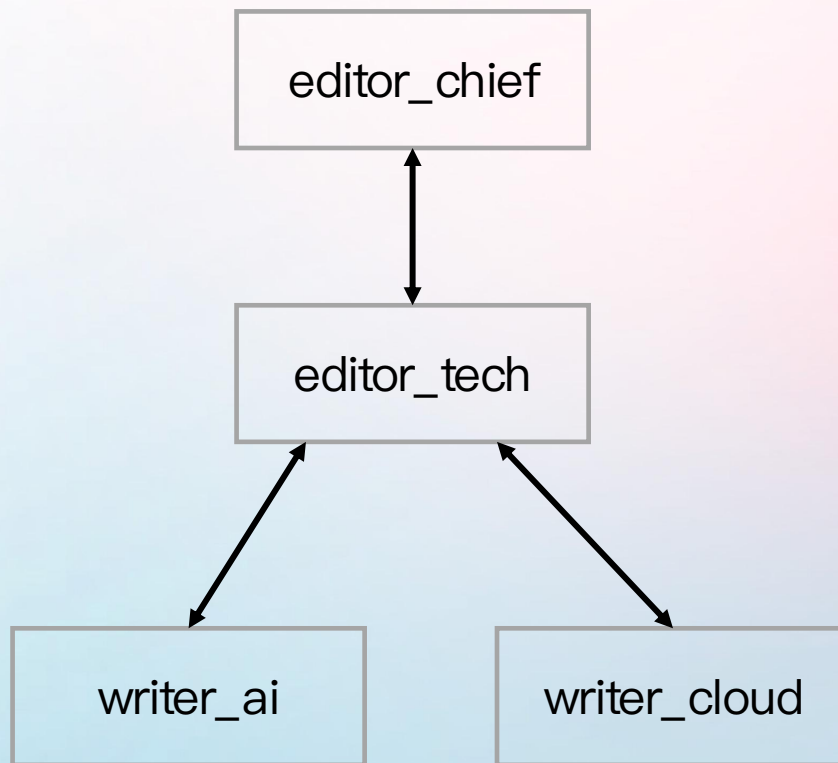
1. Create a new agent graph:

```
{
    "action": "create",
    "graph_id": "analysis_graph",
    "topology": {
        "type": "star",
        "nodes": [
            {
                "id": "central",
                "role": "coordinator",
                "system_prompt": "You are the central coordinator."
            },
            {
                "id": "agent1",
                "role": "analyzer",
                "system_prompt": "You are a data analyzer."
            }
        ],
        "edges": [
            {"from": "central", "to": "agent1"}
        ]
    }
}
```

示例：使用Agent graph创建multi agents

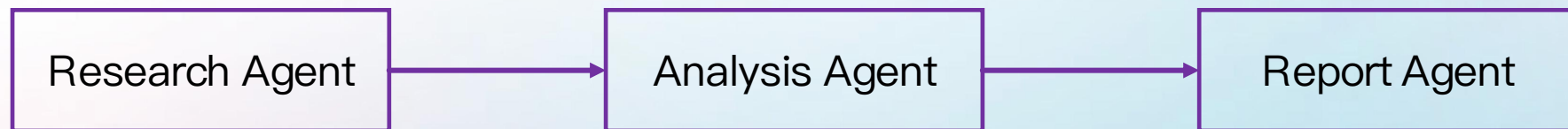
方法2: 代码指定 graph

```
# Initialize Agent with agent_graph
agent = Agent(tools=[agent_graph])
# Example: Create a content production team
agent.tool.agent_graph(
    action="create",
    graph_id="content_team",
    topology={
        "type": "hierarchical",
        "nodes": [
            {"id": "editor_chief", "role": "executive",
             "system_prompt": "You provide strategic direction and final approval."},
            {"id": "editor_tech", "role": "manager",
             "system_prompt": "You manage technical content creation."},
            {"id": "writer_ai", "role": "specialist",
             "system_prompt": "You write about AI technologies."},
            {"id": "writer_cloud", "role": "specialist",
             "system_prompt": "You write about cloud computing."}
        ],
        "edges": [
            {"from": "editor_chief", "to": "editor_tech"},
            {"from": "editor_tech", "to": "writer_ai"},
            {"from": "editor_tech", "to": "writer_cloud"},
            {"from": "writer_ai", "to": "editor_tech"},
            {"from": "writer_cloud", "to": "editor_tech"},
            {"from": "editor_tech", "to": "editor_chief"}
        ]
    }
)
```



Workflow

- 执行多步骤，有顺序和依赖条件的复杂任务
- 并行执行分支，支持指定执行优先级（1-5）
- **workflow**的状态生成**Json** 文件，持久化保存。可以暂停，恢复



示例：使用Agent workflow 创建Agentic workflow

方法1:使用自然语言动态创建workflow

自动创建workflow.json

```
# Create an agent with these tools
agent_translation = Agent(tools=[workflow], callback_handler=custom_callback_handler)

agent_translation(f"""
Create a translation workflow with the following steps:
1. A AI tech translator translate news content into Chinese.
2. Review translation and provide feedback.
3. Refine the translation based on feedback.
4. Generate 3~5 insights (depends on task complexity).
""")

text="""# AWS intros Strands Agents SDK

Many developers find agentic AI difficult,
Esther Shittu
By..."""

agent_translation(f"translate {text}")
```

```
{
  "workflow_id": "news_translation_workflow",
  "created_at": "2025-05-20T05:50:28.096395+00:00",
  "status": "completed",
  "result": {
    "translate_to_chinese": {
      "status": "completed",
      "result": [],
      "completed_at": "2025-05-20T05:51:52.689112+00:00"
    }
  }
}
```

虽然集成角度相对间接，但AWS使用独立生成式AI供应商Anthropic的模型复杂协议（MCP），"这打开了许多生态系统的大门，"Stephens说。"MCP是连接智能体工具的事实标准。"

Strands Agents可以在任何地方运行，并支持任何具有推理和工具使用能力的模型，包括Amazon Bedrock和Ollama中的模型。作为开源系统，Strands Agents包括来自社区其他成员的贡献，包括Accenture、Anthropic和OpenAI。

与此同时，OpenAI周五推出了Codex，一个基于云的软件工程智能体，可以并行处理编写功能、回答关于代码库的问题和修复错误等任务。它由codex-1提供支持，这是OpenAI的o3 LLM的一个版本，针对软件工程进行了优化，并通过强化学习进行了训练，OpenAI表示。

新智能体现在将向ChatGPT Pro、Enterprise和Team用户推出。Plus和Edu用户也将很快能够使用它。

Esther Shittu是Informa TechTarget的新闻作家和播客主持人，报道AI软件和系统。

Key Insights Generated:

关键见解

1. **AI智能体开发简化的趋势**: AWS通过Strands Agents展示了行业正在努力简化智能体AI的开发过程，让普通开发者无需深厚专业知识也能部署AI智能体。这表明AI工具正在从专家领域向主流开发者社区过渡。
2. **SDLC整合是云服务商的新战略方向**: AWS将Strands Agents与软件开发生命周期(SDLC)整合的努力，表明主要云服务提供商正扩展其价值主张，从仅提供基础设施向成为开发流程的核心部分转变。
3. **模型复杂协议(MCP)成为行业标准**: 文章指出MCP成为连接智能体工具的"事实标准"，这凸显了在快速发展的AI领域中，开放标准和互操作性变得越来越重要，可能会影响未来AI开发的格局。
4. **开源协作成为AI发展关键动力**: Strands Agents作为开源系统接受了包括Accenture、Anthropic、Meta和PwC在内的多家公司的贡献，展示了开源合作在推动AI技术发展中的关键作用。
5. **AI工具竞争加剧**: AWS推出Strands Agents的同时，OpenAI也发布了Codex，显示主要AI公司正在智能体市场展开直接竞争，这可能会加速技术创新并使开发者受益。

The entire workflow was completed successfully. The news article about AWS introducing Strands Agents SDK was translated into Chinese, reviewed for quality, refined

示例：使用Agent workflow 创建Agentic workflow

方法2: 使用代码定义workflow

```
# Connect to an MCP server using stdio transport
stdio_mcp_client = MCPClient(lambda: stdio_cli(
    StdioServerParameters(command="npx", args=
)

streamable_http_mcp_client = MCPClient(lambda:
    url="https://hapi.us-east-1.amazonaws.com/mcp",
    headers={"Authorization": "Bearer "})

# Create an agent with MCP tools
with streamable_http_mcp_client:
    with stdio_mcp_client:
        # Get the tools from the MCP server
        tools = stdio_mcp_client.list_tools_sync()
        for tool in tools:
            print(tool.tool_name, tool.tool_description)

# Sequential workflow processing
def process_workflow(topic):
    # Create specialized agents
    researcher = Agent(system_prompt="You are a research assistant. Use the provided tools to gather information.", tools=tools, context=context)
    analyst = Agent(system_prompt="You are an analyst. Synthesize the research findings into a coherent analysis.", tools=tools, context=context)
    writer = Agent(system_prompt="You are a writer. Format the analysis into a professional report.", tools=tools, context=context)

    # Step 1: Research
    research_results = researcher(f"Research the latest trends in AI agent technology.")

    # Step 2: Analysis
    analysis = analyst(f"Analyze these research results: {research_results}")

    # Step 3: Report writing
    final_report = writer(f"Create a report based on the analysis: {analysis}")

    print(f"\n----final-----:\n{final_report}")

process_workflow("AI Agent technology trends")

----final-----:
# EXECUTIVE REPORT: AI AGENT TECHNOLOGY TRENDS
## Strategic Implications for Enterprise Decision Makers

### EXECUTIVE SUMMARY
AI agent technology stands at an inflection point of extraordinary growth and business impact. Recently identified by Gartner as the #1 strategic technology trend for 2024, this emerging paradigm is reshaping the digital landscape.

### MARKET TRAJECTORY
Our analysis confirms the AI agent market is entering a phase of explosive growth:

* **Current market valuation**: $5.1 billion (2024)
* **Projected market size**: $47.1 billion by 2030
* **Compound Annual Growth Rate (CAGR)**: 44.8%
* **Enterprise adoption**: 33% of enterprise software applications will include agentic AI by 2028

This growth trajectory positions AI agents among the fastest-adopted emerging technologies in recent history, signaling a fundamental shift in how organizations will approach digital transformation.

### KEY TECHNOLOGY DEVELOPMENTS

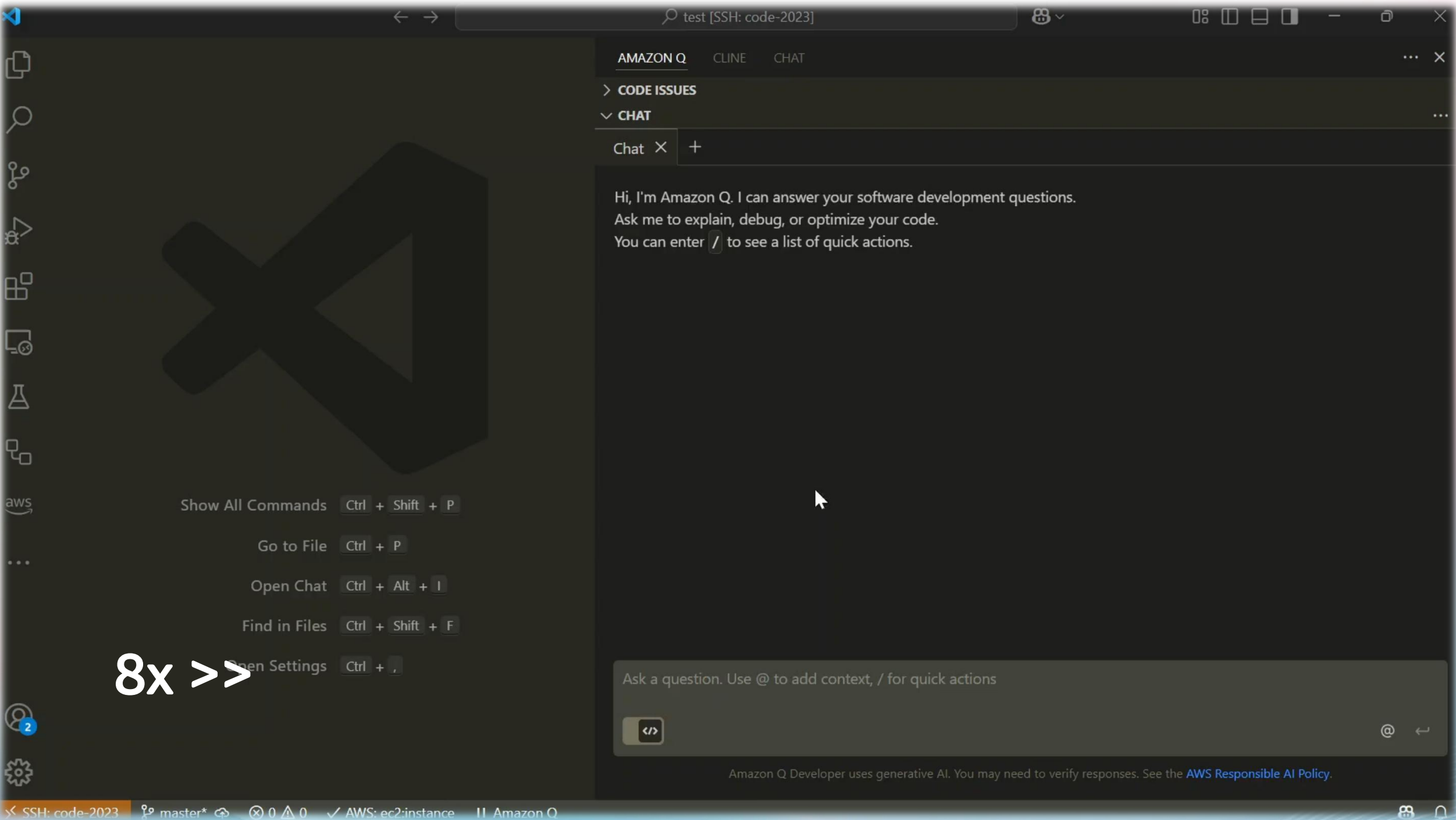
**1. Enterprise-Grade Infrastructure**
Microsoft's Azure AI Foundry Agent Service has reached general availability, providing organizations with a fully managed platform for building, deploying, and scaling AI agents.

**2. Standardization Initiatives**
The Model Context Protocol (MCP) is emerging as the critical standard for AI agent interoperability. Originally developed by Anthropic, MCP now has widespread industry adoption, enabling seamless communication between different AI systems.

**3. Agent-Ready Web Architecture**
Microsoft's introduction of NLWeb—described as "HTML for the agentic web"—represents a significant advancement in making digital content and services accessible to AI agents, paving the way for more integrated and useful agent-based applications.
```

适用于生产的规格化开发





8x >>

智能体编程的挑战



智能体编程时代的时间分配

不使用智能体编程



使用智能体编程



- 花更少时间编码
- 花更多时间规划
- 花更多时间审核

Rough idea

Vibe Coding (氛围编程)

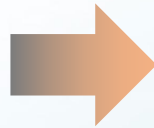
Implementation



Research
Codebase



Requirements
Clarification



Design
Solution



Implementation
Plan



Spec-driven development

案例: Spec-Driven Development

在GitHub上完成一个Pull Request

feat(cli): Add context management feature with profiles #834

Merged brandonskiser merged 57 commits into aws:main from jlhood:context 3 days ago

Conversation 37

Commits 57

Checks 10

Files changed 16



jlhood commented last week · edited

Contributor

Issue #, if available: n/a

This PR adds a context management feature to the Amazon Q Developer CLI, allowing users to maintain "sticky" context by specifying files that should always be included in chat messages.

NOTE TO REVIEWERS:

1. Highly recommend first reading about the rough idea, idea honing, and detailed designs (pasted as comments in this PR). They will help you get an understanding of this feature prior to diving into reviewing the code.
2. I actually used Q CLI to write many of the design docs as well as the implementation of this feature. I used the existing Q CLI until I had enough of the context feature working to then switch to running Q CLI with my feature and used it to write the rest of the feature and fix bugs. Using vanilla Q CLI, I had to keep telling Q to re-read the design files so it wouldn't lose context on the feature being built. Once I had a version of Q running with the context feature, I was able to always keep the relevant design/implementation files in context so I didn't have to keep reminding Q about them. So I did a fair amount of manual testing/verification of the feature as I built it. 🙄🙄

Features

- Add, remove, and clear context files globally or per profile
- Support for glob patterns to include multiple files
- Profile management (create, delete, switch, rename)
- Visual indicator in prompt showing active profile
- CLI flag for specifying profile at startup (--profile)
- Support for adding non-existent files with --force flag
- Comprehensive error handling

Implementation Details

Amazon Q Developer CLI 添加一个“上下文管理”的新功能功能（使用Amazon Q CLI）

此贡献者不会 Rust 语言，也不熟悉Amazon Q Developer CLI 的代码

全部文档和代码由 Amazon Q Developer 2天完成，开发者预计手写需要4周左右

<https://github.com/aws/amazon-q-developer-cli/pull/834>

jlhood commented 3 weeks ago

Contributor Author ...

Rough Idea

Needs

I have "cheat sheets" I've created to help Q understand Amazon internals and be able to interact with different systems like internal search and internal code browser. Some cheat sheets are general (README.md and development.md) and some are very specific (golang.md). While I have some success with telling Q to read these files, I have to keep telling Q to read and re-read them periodically as they fall outside the context window. I don't have visibility into when this is going to happen so I kind of have to guess or I just sort of notice that it's happening based on Q's behavior suddenly taking a turn for the worse.

Given context size is limited, I don't always need all cheat sheets for every given task. For example, I only need Q to read golang.md when working on a Go package. Also, there are certain docs that I want in the context only while working on a specific task. For example, I have a convention where I'm creating a folder named after a task id and using it to store planning docs (research summary, implementation plan, todo.md, etc). I want these in context just while working on that specific task.

I'd like all of these docs to be in source control, but the location can vary given we work across many repos (packages) within Amazon.

UX Idea

I'm picturing an experience where I can select specific context files/folders both globally as well as for a given session. A UX idea I've been playing around with is modeling an experience based on git as I think there are clear parallels that you'll see from the commands I'm about to define.

We can add a new command /context to Q CLI that allows you to manage "sticky" context, i.e., files that should always be included as context in every chat message request. /context supports the following subcommands (final names TBD):

1. show - show information about the current context (current profile and contents of that profile)
2. add - add one or more files/folders to the current context
3. rm - remove one or more files/folders from the current context
4. profile - list context profiles
5. switch - switch to a different context profile

Some concepts to understand:

1. Profile - a profile is just a name for a group of context files. The default context profile is called "default".
2. Global context - You can add files/folders to context "globally" meaning it's automatically applied to all profiles. Global context can be modified by using the --global flag with add/rm commands.

Rough Idea

实际上，绝大部分的需求都来自于一个
“rough idea”/粗略的想法

Rough idea



Research
Codebase



Requirements
Clarification



Design
Solution



Implementation
Plan



Implementation

Research Codebase

- > What is this codebase?
- > Do more research on the q chat [implementation](#) and give me a summary
- > Create a directory called planning and write this summary information to a file called codebase.md
- > write a file in the planning folder called slashcommand.md covering how slash commands are implemented. Include enough detail so a developer could implement a new slash command using this information

Research Codebase

Amazon Q Developer CLI Codebase Summary

Overview

The **Amazon Q Developer CLI** is part of a monorepo that houses the core code for the Amazon Q Developer desktop application and command-line interface. Amazon Q Developer is an AI assistant built by AWS to help developers with various tasks.

Key Components

1. **q_cli**: The main CLI tool that allows users to interact with Amazon Q Developer from the command line
2. **fig_desktop**: The Rust desktop application that uses tao/wry for windowing and webviews
3. **Web Applications**: React apps for autocomplete functionality and dashboard interface
4. **IDE Extensions**: VSCode, JetBrains, and GNOME extensions

🔗 Project Structure

- `crates/` - Contains all internal Rust crates
- `packages/` - Contains all internal npm packages
- `proto/` - Protocol buffer message specifications for inter-process communication
- `extensions/` - IDE extensions
- `build-scripts/` - Python scripts for building, signing, and testing
- `tests/` - Integration tests

Amazon Q Chat Implementation

Amazon Q Chat implementation

Slash Command Implementation Guide

This document explains how slash commands are implemented in the Amazon Q Developer CLI chat interface and provides guidance on how to implement new slash commands.

Overview

Slash commands in the Amazon Q chat interface provide special functionality that can be triggered by typing a command that starts with a forward slash (/). Examples of existing slash commands include `/help`, `/clear`, `/acceptall`, and `/quit`.

Implementation Architecture

The slash command system is primarily implemented in the following files:

1. `src/cli/chat/command.rs` - Defines the `Command` enum and parsing logic
2. `src/cli/chat/mod.rs` - Contains the main chat loop that processes commands
3. `src/cli/chat/prompt.rs` - Implements command completion for slash commands

Command Parsing

Slash commands are parsed in the `Command::parse` method in `command.rs`. Here's how it works:

```
pub fn parse(input: &str) -> Result<Self, String> {  
    let input = input.trim();  
  
    if let Some(command) = input.strip_prefix("/") {  
        if let Some(command) = TUBING.parse_command(command) {  
            return Ok(TUBING.parse_command(command));  
        }  
    }  
}
```

Rough idea



Research
Codebase



Requirements
Clarification



Design
Solution



Implementation
Plan

Implementation



Requirements Clarification

> 阅读 rough-idea.md。一个问题一个问题地问人类，这样我们就能为这个想法制定一个全面的、循序渐进的规范.....

Context Management Feature Specification Development

Q&A Summary

Q1: What specific problems are you trying to solve with the context management feature that aren't addressed by the current implementation of Amazon Q Developer CLI?

A1: Today, Q CLI chat maintains a context window, but older chat messages can age off the context window, leading to Q "forgetting" important things like the current task or goal you're trying to drive toward. This feature addresses this by allowing you to specify files that will always be passed as context on every chat message.

Q2: How should the context management feature handle file size limits and prioritization when multiple context files are added?

A2: The file content should be added to the context in order. Global context file content should be added first, followed by alias-specific context file content.

Q3: How should the system handle cases where the total size of all context files exceeds the maximum context window size?

A3: Eventually, the show command should include the percentage of total context that's taken up by the files, but this feature can be deferred for now since it's not clear how much context is actually available.

Q4: How should the context management feature handle file paths and glob patterns?

Q4: How should the context management feature handle file paths and glob patterns?

deferred for now since it's not clear how much context is actually available.

A3: Eventually, the show command should include the percentage of total context that's taken up by the files, but this feature can be deferred for now since it's not clear how much context is actually available.

Q3: How should the system handle cases where the total size of all context files exceeds the maximum context window size?

Rough idea



Research
Codebase



Requirements
Clarification



Design
Solution



Implementation
Plan



Implementation

Design Solution

- 根据目前所有的规划文档, 创建一份详细的设计文档, 以便开发人员可以立即开始实施.

Context Management Feature - Detailed Design Document

Overview

This document provides a comprehensive design for implementing the context management feature in the Amazon Q Developer CLI. This feature allows users to maintain "sticky" context by specifying files that should always be included in the chat context, organized through profiles.

Table of Contents

1. [Feature Requirements](#)
2. [Architecture](#)
3. [Data Structures](#)
4. [File Storage](#)
5. [Command Interface](#)
6. [Implementation Plan](#)
7. [Testing Strategy](#)
8. [Future Enhancements](#)

Feature Requirements

The context management feature must:

1. Allow users to add files/folders to the context globally or for specific profiles
2. Support glob patterns for file selection

Rough idea



Research
Codebase



Requirements
Clarification



Design
Solution



Implementation
Plan

Implementation



Implementation Plan

- 将实施计划转换为代码生成 LLM 的一系列提示词
- 该 LLM 将以测试驱动的方式实施每个步骤...

Context Management Feature Implementation Prompt Plan

This document outlines a series of prompts for implementing the context management feature in a test-driven manner. Each prompt builds on the previous ones, ensuring incremental progress and early testing.

Prompt 1: Create the Core Data Structures

Create the core data structures for the context management feature. Implement the `ContextConfig` struct for storing 

Specifically:

1. Create a new file `src/cli/chat/context.rs` with the `ContextConfig` and `ContextManager` structs
2. Implement serialization/deserialization for `ContextConfig` using serde
3. Add methods to initialize the `ContextManager` with default paths
4. Implement methods to load and save configurations from/to JSON files
5. Write unit tests for these core functions

Focus on the data structures and file operations without implementing the command interface yet.

Prompt 2: Implement Path Management Functions

Implement the path management functions for the `ContextManager`. These functions should handle adding, removing, and 

Specifically:

1. Implement `add\_paths` method to add paths to global or profile context
2. Implement `remove\_paths` method to remove paths from global or profile context
3. Implement `clear` method to clear all paths from global or profile context

3. Implement `clear` method to clear all paths from global or profile context
3. Implement `remove\_paths` method to remove paths from global or profile context
1. Implement `add\_paths` method to add paths to global or profile context
Specifically:

Implement the path management functions for the `ContextManager`. These functions should handle adding, removing, and 

Implement the path management functions for the `ContextManager`.

Rough idea



Research
Codebase



Requirements
Clarification



Design
Solution



Implementation
Plan



Implementation

Implementation

- 实施提示计划中的提示词 1
- 更新 todo.md 的进度。

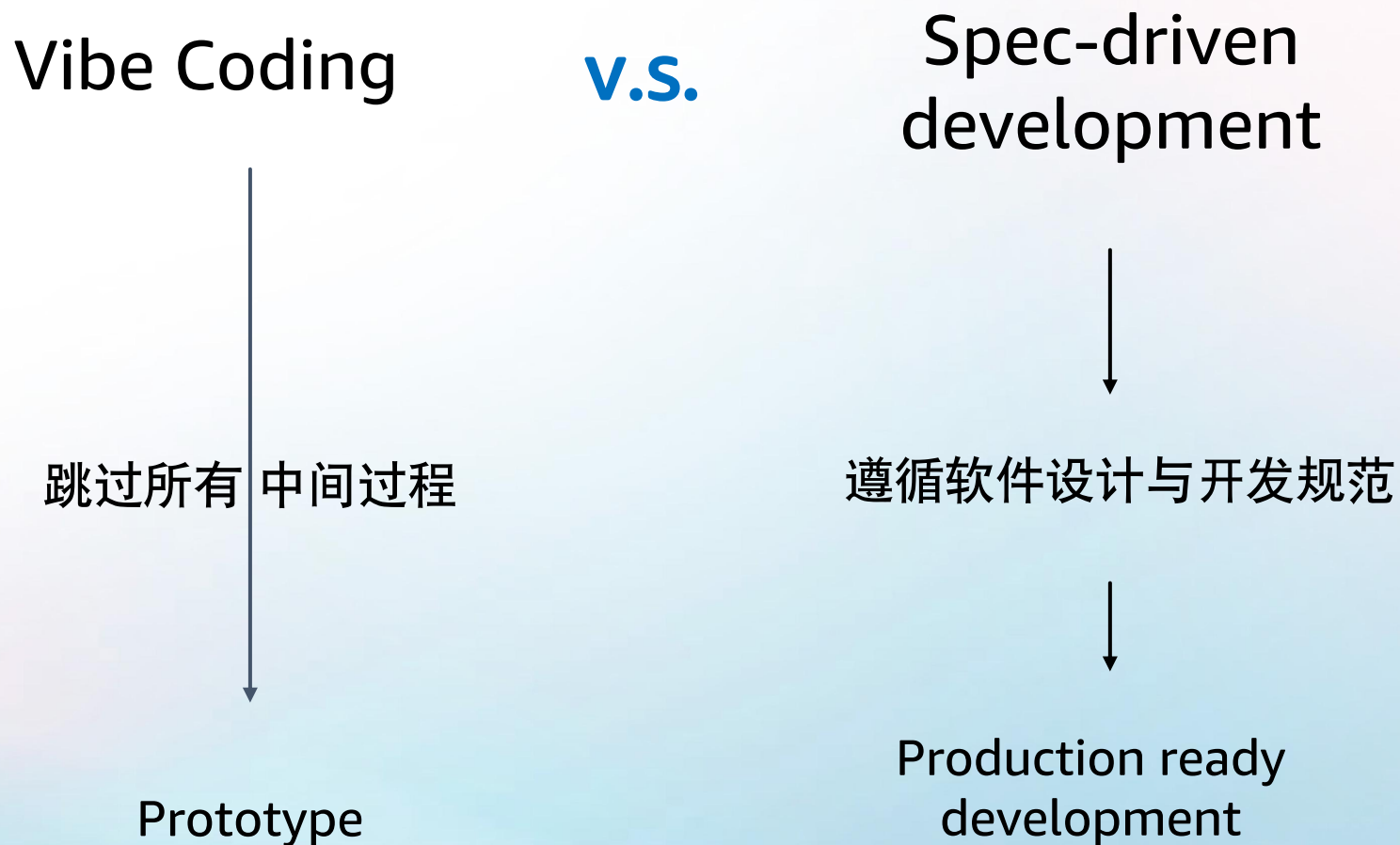
<https://github.com/aws/amazon-q-developer-cli/pull/834>

Execute build	594f1dc
Trigger build (aws#33)	1d13a31
Trigger Build (aws#34)	0f8a26b
docs: planning context feature	ce89d90
Rename alias -> profile	b41cb50
Detailed design	cbe2e96
Add todo list	fa95de5
Updated detailed design	8a7355e
Remove todo	53eaa10
Add planning subdirs	22f2c24
Add implementation plan	e4b1267
implement prompt 1	a64ce6e
implement prompt 2	243b2ca
Fix context.rs tests by adding module to chat/mod.rs and removing int...	493ea27
Implement profile management functions for context feature	733a635
Fix test visibility for context manager	aa9f9ed
Fix context management implementation issues: add clear method and fi...	156b7de
Update profile name validation to require alphanumeric first character	76b64d2
Implement context file processing (prompt 4)	d2d7bc6
Implement prompt 5: Extend the Command Enum and Parser for context ma...	0499c98
Implement command completion for context management feature	286e04f
Update help text with context management commands	f360000
Implement prompt 8: Integrate with ConversationState	0ca6bf6

现代开发范式

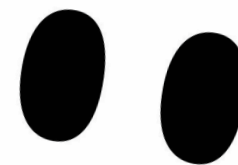


生成式 AI 下的新型软件开发范式



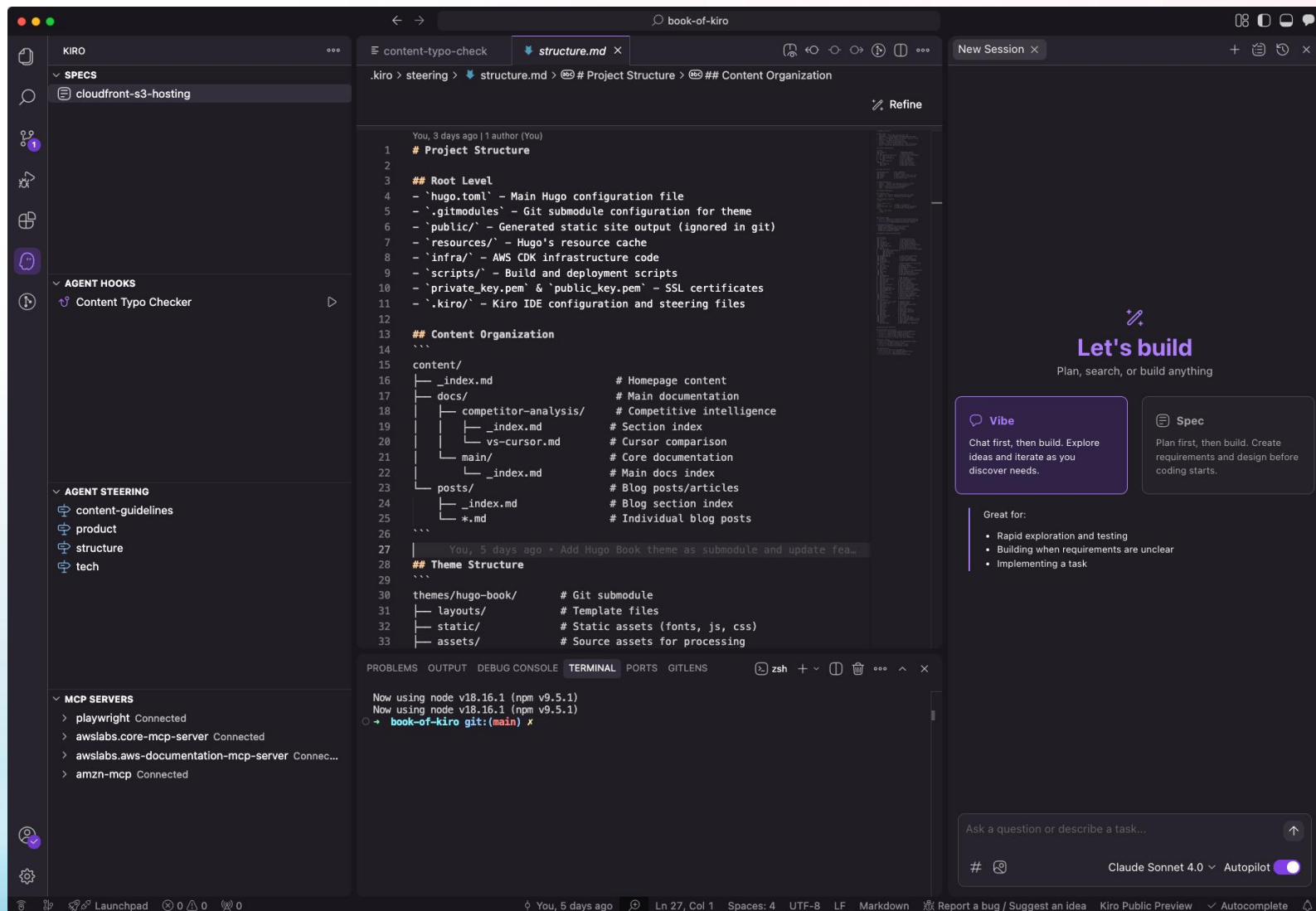
KIRO

从原型到生产的 AI IDE



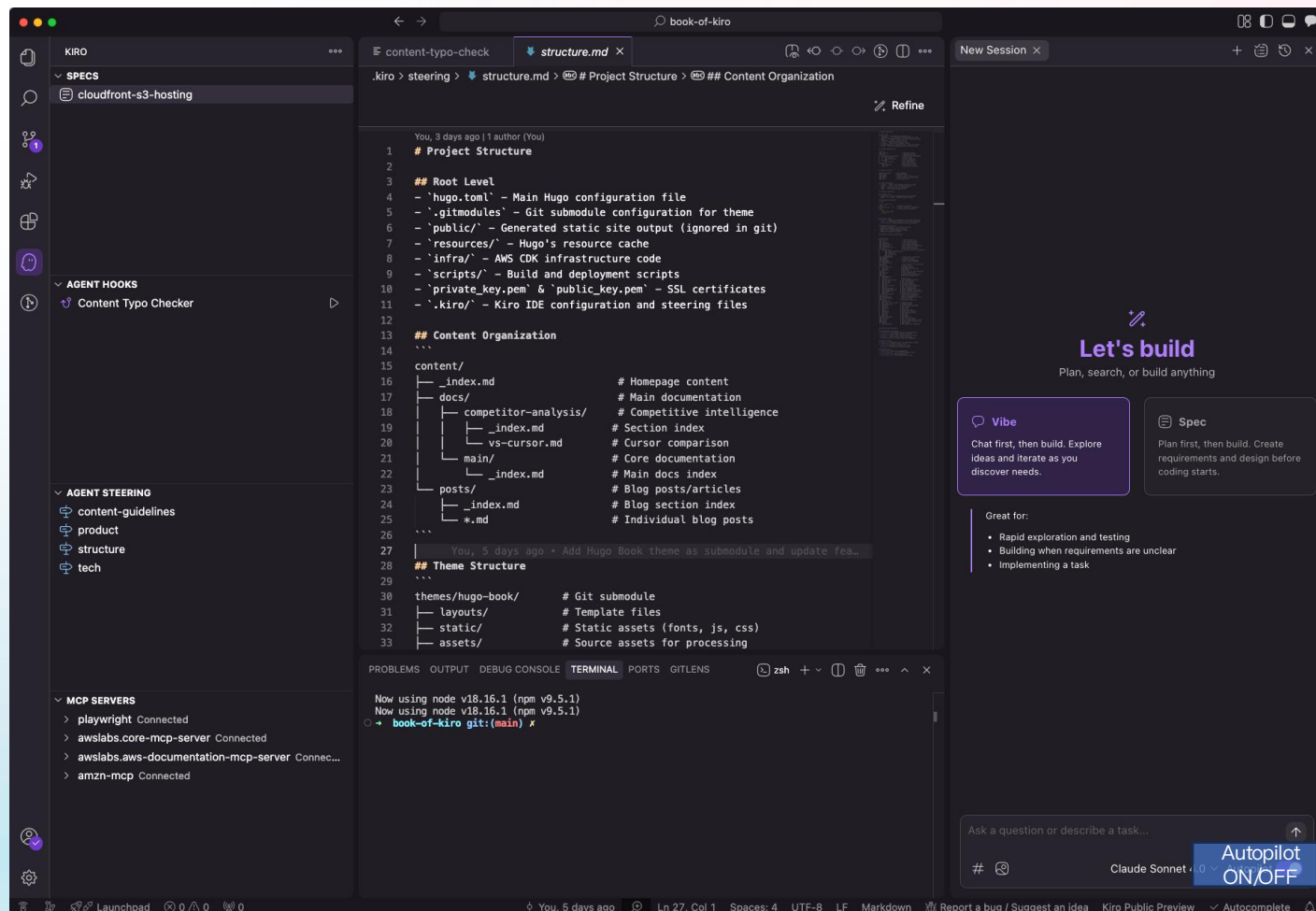
构建方式: Vibe & Spec

- **Vibe:** Vibe 是 Kiro 中以互动问答为核心的会话模式，专为快速提问、解释说明以及通过更加对话化的方式构建项目而设计。
- **Spec:** Spec 在 Kiro 中为您提供结构化的方法来处理复杂的开发任务，将软件开发流程正规化。它能够将高层次的想法转化为详细的实施计划，并进行系统化执行和清晰的跟踪。



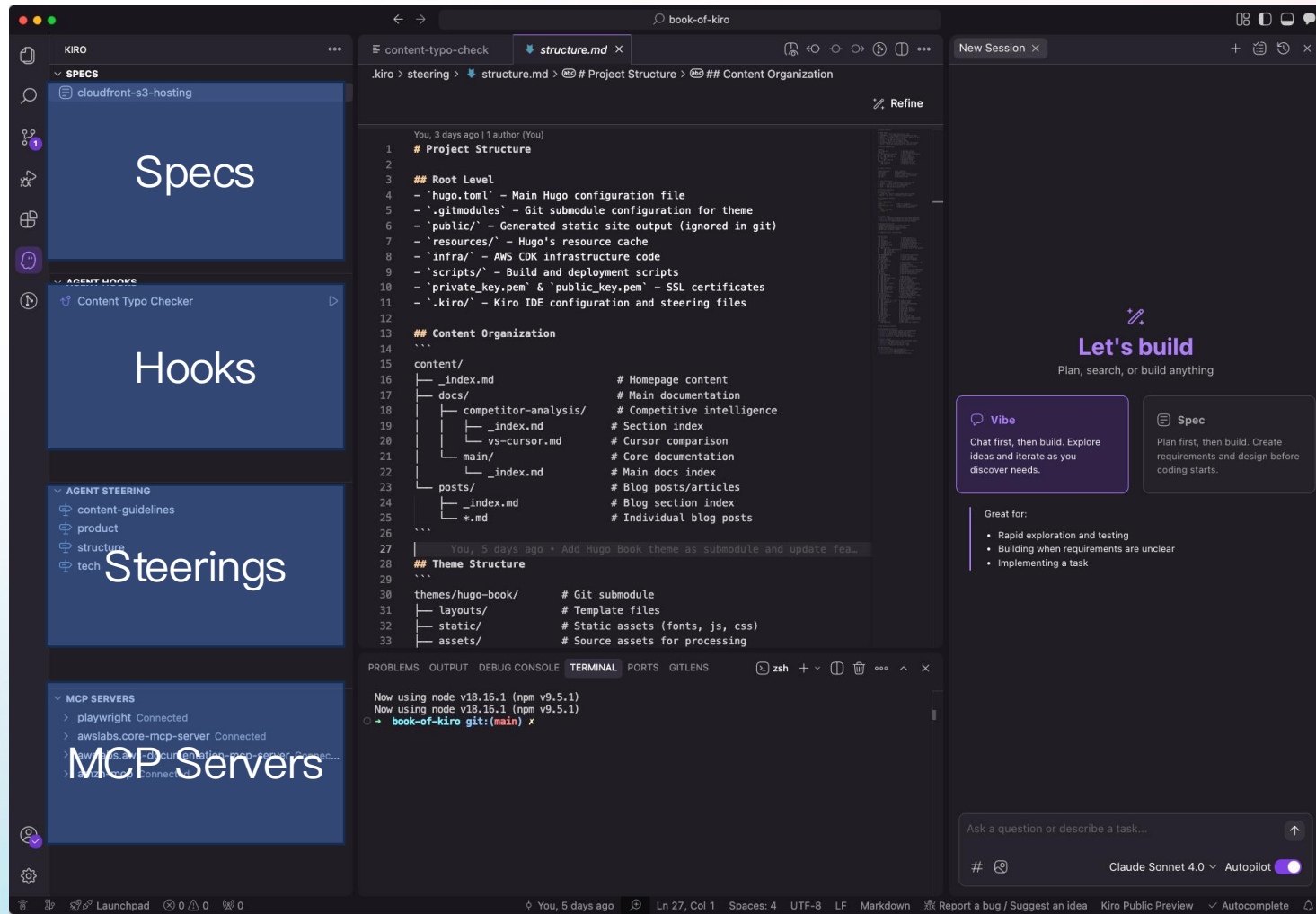
Agent 模式

- **Autopilot:** Kiro 能够自主完成端到端的任务。它可以创建文件、在多个位置修改代码、运行命令，并自行做出架构决策，无需在每个步骤都征求批准。你可以通过查看所有变更、随时回滚或中断执行来保持控制。
- **Supervised:** Kiro 会展示每个拟议的操作并等待你的批准后再继续。你可以清楚地看到 Kiro 想要做出哪些变更，并可以接受、拒绝或修改这些变更。这种模式将复杂任务分解为可管理的步骤，并提供明确的反馈检查点。



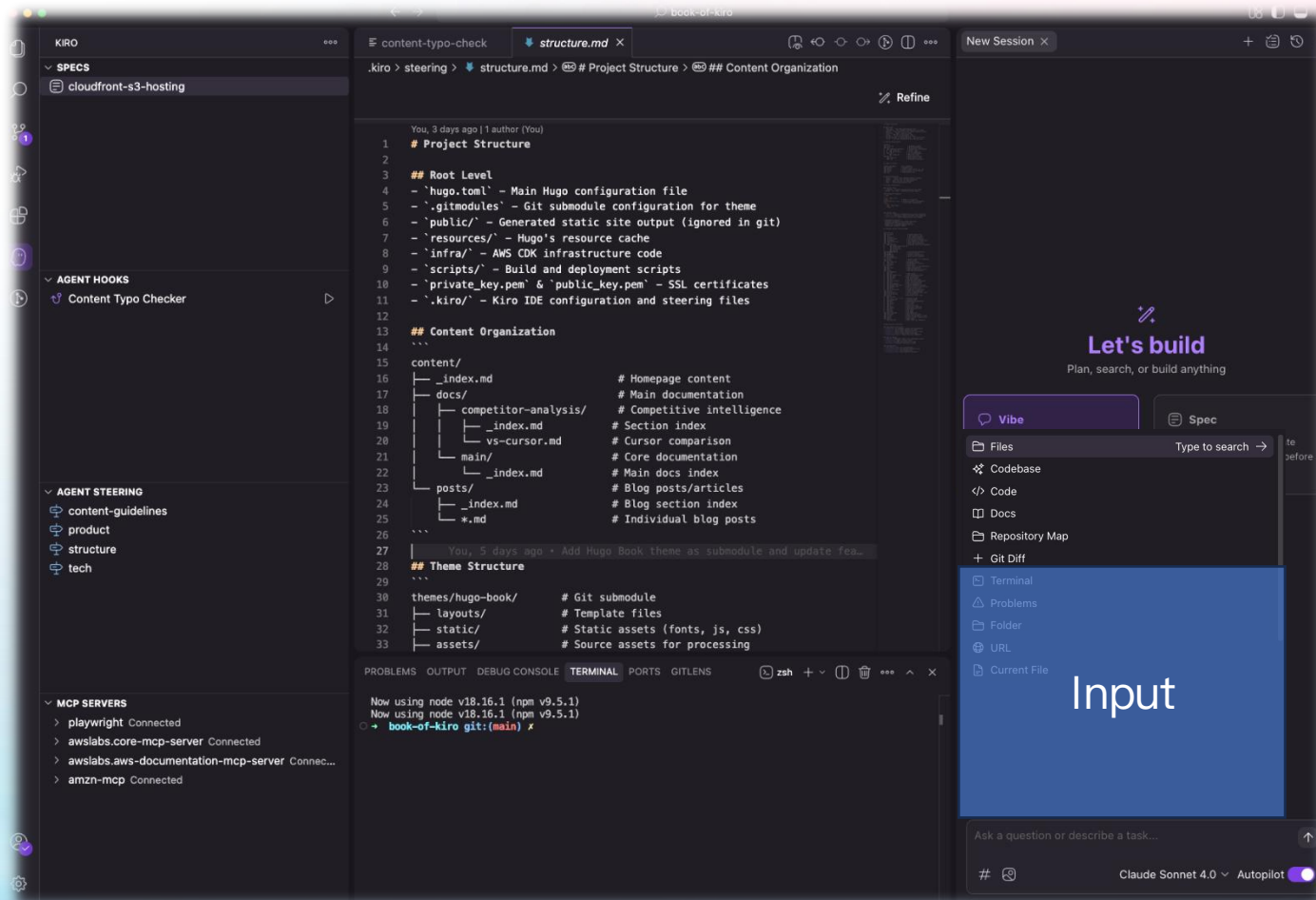
Kiro 面板

- **Specs:** 管理代码生成过程中的 Specs 文档，任务有迹可循，团队协作更方便
- **Hooks:** 管理的 Agent Hooks 设置，让团队协作中的成员保持项目规范统一
- **Steering:** 通过 markdown 文件为 Kiro 提供关于您项目的知识，指导文件确保 Kiro 始终遵循您已建立的模式、库和标准。
- **MCP Servers:** 通过 MCP 来扩展 Kiro 的功能，为其提供更多工具和上下文信息。



上下文 (Context)

- Codebase: 当前项目的代码索引
- Files/Folders: 文件或文件夹
- Code: 代码片段
- Docs: 常用开发文档
- Git Diff: 当前的 Git Diff
- Terminals: 命令行中的内容
- Problems: 当前打开文件中被 IDE 自动检测出来的问题
- URL: 给定一个 URL 的网页内容
- Steering: Kiro 的 Steering 文件
- MCP: 调用某个 MCP



Get started with Kiro for free



Kiro.dev



谢谢

